

Individual Project Technical Report

Oil Palm Fruit Ripeness Level Classification Using a Convolutional Neural Network (CNN)

Name:	Goh Tse Thing
Student ID:	BAI_A2009F-2605002
Lecturer:	Dr. Ng Choon Ching
Module:	BAI13143 Neural Network Basics
Semester:	Sept 2026
Due Date:	11/11/2026

Student's Name:	Goh Tse Thing	Student's ID:	BAI_A2009F-2605002
Module Code & Name:	BAI13143 Neural Network Basics		
Assignment/ Project Title:	Oil Palm Fruit Ripeness Level Classification Using a Convolutional Neural Network (CNN)		
Lecturer's Name:	Dr. Ng Choon Ching		
Semester/ Year:	Sept 2026		

I hereby declare that the work in this assignment/project was carried out in accordance with the regulations of Raffles University. It is original and is the results of my own work, unless
otherwise indicated or acknowledged as referenced work.

I certify that the statements I have attested to above have been made in a good faith, true and correct. I acknowledge that any misconduct or violation of academic integrity policies will result in disciplinary action and may affect my academic standing in the institution.

8/11/2026

....

Date Student's Signature

Table of Contents

1. Introduction 4
2. Dataset and Preparation 4
   2.1. Dataset 4
   2.2. Data checking and splitting 5
   2.3. Image preprocessing 6
3. Network Design 6
4. Training and Experiments 7
   4.1. Training settings 7
   4.2. Controlled experiments 8
   4.3. Overfitting and Underfitting Analysis 10
5. Evaluation 10
   5.1. Final test results 10
   5.2. Per-class results 11
5.3. Confusion matrix and mistakes 11
6. Docker Setup 13
   6.1. Training container 13
   6.2. Testing container 14
   6.3. Cross-Machine Verification 15
7. Reflection 16
8. References 16

1. Introduction

Oil palm is an important crop in Malaysia and other countries. The ripeness of oil palm fruit would affect the harvesting decisions and palm oil quality. Farmers usually would check the fruit color and appearance to decide whether the fruit is ready for harvesting. But this process also can be hard because some ripeness levels look very similar. Not for that, the lighting and camera angles also can affect how the fruit appears.

A Convolutional Neural Network (CNN) can help to learn useful features from the images such as color, shape and texture. Then, It can use these features to classify new images.

The goal of this project was to build a CNN that can help to classify the oil palm fruit into five categories such as Immature, Partially Ripe, Fully Ripe, Overripe and Decayed.

The input of the CNN is a 256x256 RGB image of an oil palm fruit branch. The output is one of the five ripeness classes. The success target of this project was to achieve at least 85% test accuracy.

The objectives were to prepare a suitable dataset, design a CNN from scratch, compare different training settings, evaluate the final model and use the Docker to run training and testing in separate environments.

The project used PyTorch for model development and Docker for deployment.

2. Dataset and Preparation

2.1. Dataset

This project used An Ordinal Dataset for Ripeness Level Classification in Oil Palm Fruit Quality Grading, available from Mendeley Data [1].

The dataset contains 4,728 RGB images and each with a resolution of 256 x 256 pixels. The images were divided into five classes.

Table 1: Dataset class distribution

Class	Images
Immature	758
Partially Ripe	1,405
Fully Ripe	1,489
Overripe	665

Decayed	411
Total	**4,728**

The dataset was not balanced. Fully Ripe had the most images while the Decayed had the fewest. This could affect the model because it received more the training example from some classes than others.

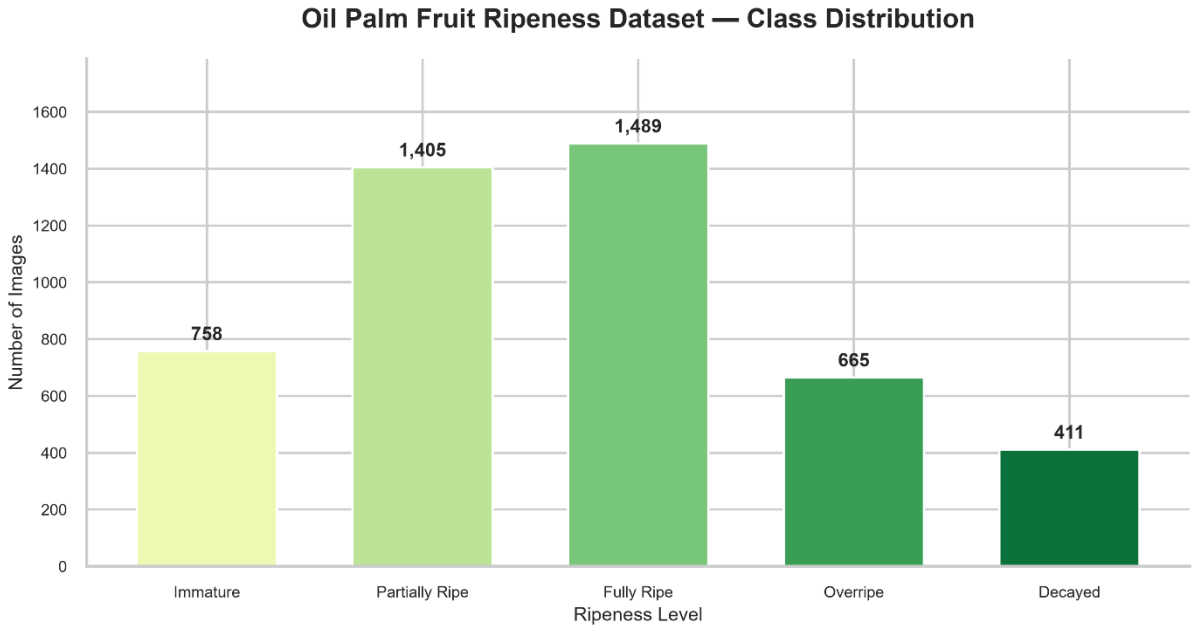


Figure 1: Class contribution of Oil Palm Fruit Ripeness Dataset

2.2. Data checking and splitting

Before training, I checked the dataset for missing files, unreadable images and duplicate images.

All images could be opened successfully. One exact duplicate pair was found in the training set which is Decayed321.jpg and Decayed322.jpg. I excluded the Decayed322.jpg from training without deleting the original file.

I used the official dataset split files instead of creating a new random split.

Table 2: Dataset split

Split	**Images used**
Training	3305
Validation	704
Testing	718

Total	**4727**

The training set was used to update the CNN weights. The validation set was used in order to compare experiments and select the best model. The test set was kept separate until the final evaluation.

2.3. Image preprocessing

I created a custom PyTorch Dataset class to load the images and their labels.

Each image was resized to 256 x 256 pixels, converted into a tensor and normalized using a mean and standard deviation of 0.5 for each RGB channel.

Normalization can help to keep the input values on a consistent scale. The same preprocessing steps were used during training, validation and testing.

I also created sample image and brightness plots in order to understand the dataset before training.

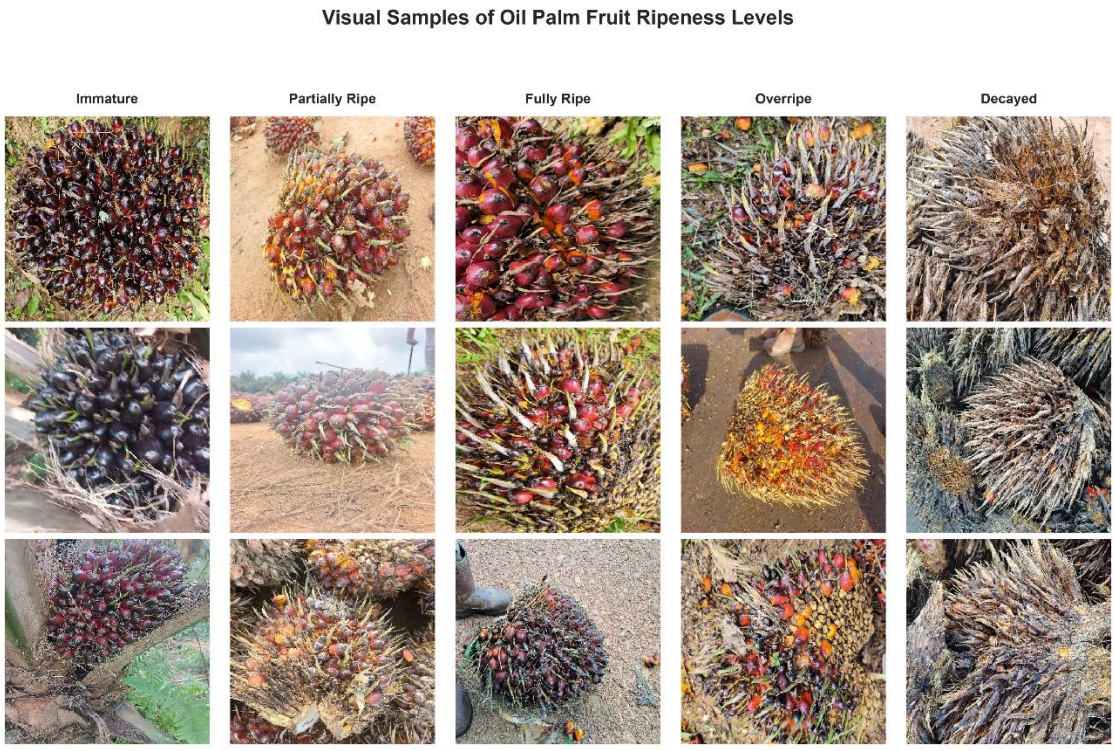


Figure 2: Sample images from the five classes

3. Network Design

I designed a CNN from scratch instead of using a pre-trained model. The network contains three convolutional blocks and two fully connected layer.

Table 3: CNN architecture

Layer	Configuration
Input	3 x 256 x 256
Conv Block 1	3 → 16 channels, ReLU, Max Pooling
Conv Block 2	16 → 32 channels, ReLU, Max Pooling
Conv Block 3	32 → 64 channels, ReLU, Max Pooling
Adaptive Average Pooling	4 x 4
Flatten	1,024 features
Fully Connected	1,024 -> 128, ReLU
Dropout	0.3 or 0.5
Output	128 → 5 classes

The CNN has 155, 429 trainable parameters.

The convolutional layers would learn the image features. ReLU can help the network to learn more about the complex patterns while the max pooling can help to reduce the feature-map size. Adaptive average pooling would produce a fixed number of features before classification.

The fully connected layers use these features to produce five output values called logits. The class with the highest outputs is selected as the prediction.

Dropout was included in order to reduce the overfitting. During the training, it randomly disables some hidden activations. During evaluation, dropout is disabled by calling model.eval().

4. Training and Experiments

4.1. Training settings

I trained the CNN using PyTorch with the following settings:

Table 4: Training configuration

Setting	Value
Optimizer	Adam
Loss function	CrossEntropyLoss
Batch size	16
Epochs	20

Random seed	42
Input size	256 x 256
Number of classes	5

Adam was used to update the model weights. Cross-entropy loss was suitable because each image belonged to one of five classes.

During the training, I recorded the training and validation loss and accuracy for every epoch. I saved the checkpoint with the lowest validation loss instead on automatically using the final epoch.

4.2. Controlled experiments

I conducted one baseline experiment and two controlled experiments. I changed the only one hyperparameter at a time so the results could be compared fairly.

Table 5: Experiment results

Experiment	Learning rate	Dropout	Best Validation Loss	Validation Accuracy
Baseline	0.001	0.3	0.5355	83.10%
Experiment 1	0.0005	0.3	0.5840	78.41%
Experiment 2	0.001	0.5	0.5337	82.67%

Validation accuracy at the checkpoint is selected by lowest validation loss.

The baseline model performed well. Its best validation loss was 0.5355 at epoch 19.

In Experiment 1, I reduced the learning rate from 0.001 to 0.0005. The validation loss increased to 0.5840. This suggests that the smaller learning rate did not help the model enough within 20 epochs.

In Experiment 2, I increase the dropout from 0.3 to 0.5 and kept the original learning rate. This experiment achieved the lowest validation loss of 0.5337 at epoch 19.

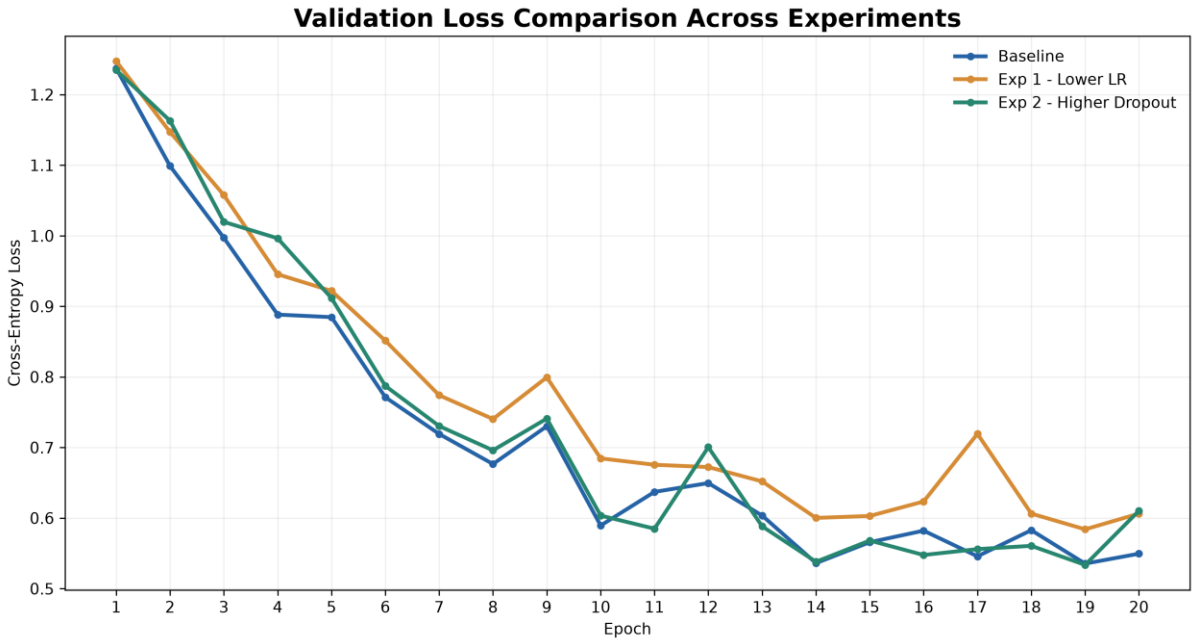


Figure 3: Validation Loss Comparison

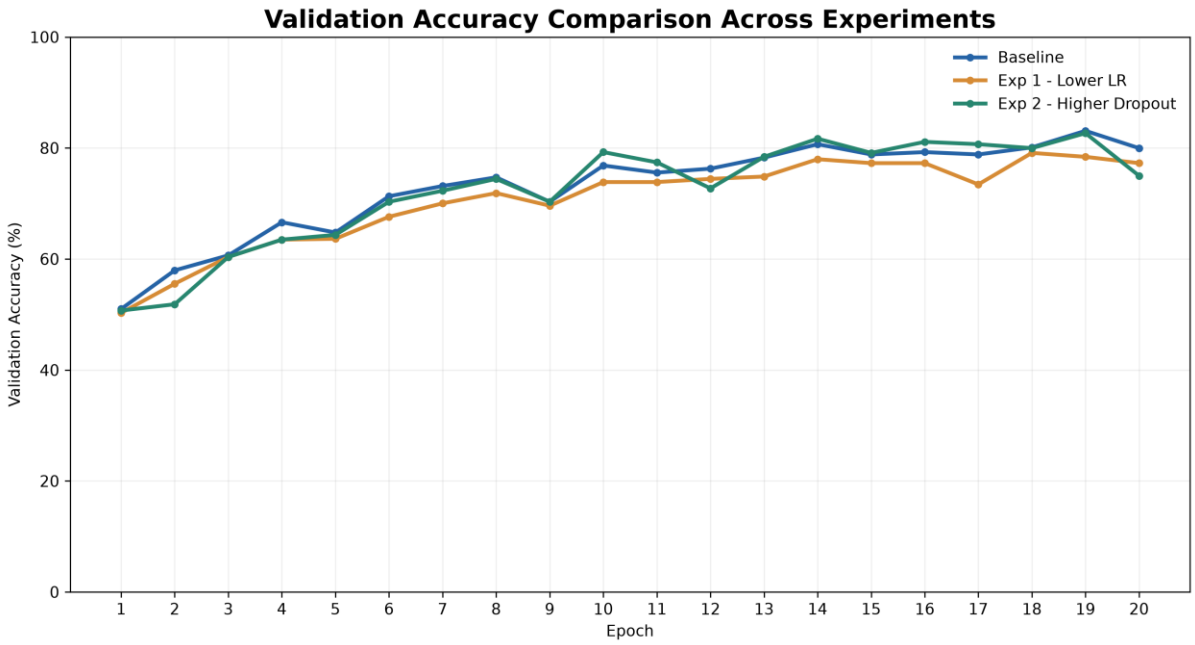


Figure 4: Validation accuracy comparison

I selected the Experiment 2 for final testing because the lowest validation loss was my selection rule.

However, its validation loss was only slightly better than the baseline. Thus, I cannot conclude the increasing dropout always improves the CNN performance.

The test set was not used to choose the model or adjust hyperparameters.

4.3. Overfitting and Underfitting Analysis

I compared the training and validation loss curves in order to check the overfitting and underfitting.

For the baseline experiment, the training loss was approximately 0.347 at epoch 20 while the validation loss was approximately 0.550. This gap suggests that is mild overfitting because the model performed better on the training data than on the validation data.

For the Experiment 2, the validation loss increased from approximately 0.534 at epoch 19 to 0.610 at epoch 20. This shows that the validation performance did not improve continuously.

There was no clear evidence of severe underfitting because the model achieved relatively good validation accuracy. Overall, the results suggest mild overfitting and some fluctuations in validation performance.

The best checkpoint was selected using the lowest validation loss rather than the final epoch.

5. Evaluation

5.1. Final test results

After selecting the Experiment 2, I evaluated its saved checkpoint on the official test set of 718 images.

The CNN correctly classified 589 images and incorrectly classified 129 images.

Table 6:Final Test performance

Metric	**Result**
Accuracy	82.03%
Macro Precision	84.46%
Macro Recall	82.41%
Macro F1-score	83.28%
Weighted F1-score	82.04%
Correct predictions	589
Incorrect predictions	129

Accuracy measures the overall percentage of the correct predictions. Precision shows how many predictions for a class were correct. Recall shows how many actual images of a class were correctly found. F1-score combines precision and recall.

I used macro F1-score because it gives equal importance to all the five classes including the smaller classes.

As a simple comparison, always predicting the larger test class, Fully Ripe would give only 31.34% accuracy. This shows that the CNN learned more useful information than a majority-class prediction method.

However, the final accuracy was below the project's 85% target by approximately 2.97 percentage points.

5.2. Per-class results

Table 7: Classification report

Class	Precision	Recall	F1-score
Immature	89.52%	81.74%	85.45%
Partially Ripe	77.43%	82.16%	79.73%
Fully Ripe	79.66%	83.56%	81.56%
Overripe	85.06%	72.55%	78.31%
Decayed	90.62%	92.06%	91.34%

Decayed had the highest F1-score at 91.34%. Overripe had the lowest recall at 72.55% which meaning that's many actual Overripe images were predicted as other classes.

5.3. Confusion matrix and mistakes

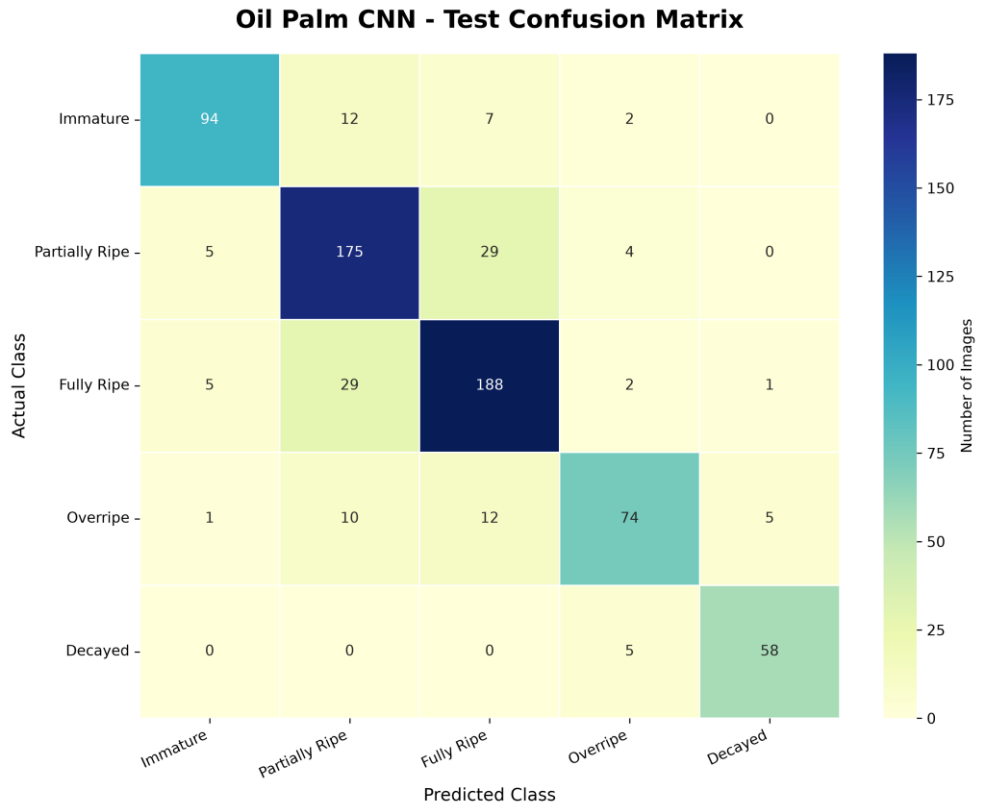


Figure 5: Confusion matrix

The confusion matrix showed that the biggest problem was distinguishing Partially Ripe from Full Ripe.

There were 29 Partially Ripe images predicted as Fully Ripe and 29 Fully Ripe images predicted as Partially Ripe. These 58 mistakes made up about 45% of all 129 incorrect predictions.

A possible reason is that neighboring ripeness stages have similar fruit colors and textures.

I also inspected five incorrect prediction. All the five examples in the saved gallery were Decayed images that predicted as Overripe. These mistakes suggest that the CNN sometimes would struggles to distinguish very ripe fruit from fruit showing signs of decay.



Figure 6: Five incorrect prediction

These five examples show on type of error. They do not represent every mistake made by the model.

Overall, the results show that the CNN can classify many images correctly but it still has difficulty with visually similar ripeness levels.

6. Docker Setup

I used Docker to run the CNN in environments that do not need to depend on my local Conda installation.

I created two Separate Docker Images:

- oilpalm-train:1.0 for training
- oilpalm-test:1.0 for testing and prediction

The images were built using the Dockerfile.train and Dockerfile.test. Both used a PyTorch 2.14.0 base image with CUDA 12.6 support.



IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
hello-world:latest	c567b231f656	25.9kB	9.49kB	
oilpalm-test:1.0	b31b223ffb9e	12GB	4.07GB	
oilpalm-train:1.0	9d6686b7d600	11.7GB	4.01GB	

Figure 7: Training and testing Docker images

6.1. Training container

The training container used the laptop GPU and completed a separate 20-epoch baseline training run.

Its best validation loss was 0.5188 at epoch 17 with 81.25% validation accuracy at that checkpoint.

I used the Docker bind mounts [2] to let the container read the dataset and save the result to the Windows computer.

The generated files included baseline_best.pt and baseline_history.csv. They are still available after the contained stopped.

This was a separate baseline training run. It was not the Experiment 2 checkpoint used for the official final evaluation.

```
(oilpalm) PS C:\degree\Sem_2\NEURAL NETWORK BASICS\Offline_Assignment> Get-ChildItem .\docker_artifacts\models\
>> Get-ChildItem .\docker_artifacts\outputs\training\


    Directory: C:\degree\Sem_2\NEURAL NETWORK BASICS\Offline_Assignment\docker_artifacts\models


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----         11/9/2026  12:15 AM         626533 baseline_best.pt


    Directory: C:\degree\Sem_2\NEURAL NETWORK BASICS\Offline_Assignment\docker_artifacts\outputs\training


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
-a----         11/9/2026  12:17 AM           1618 baseline_history.csv
```

Figure 8: Model checkpoint and training history saved by the Docker training container.

6.2. Testing container

The testing container successfully loaded the selected model, exp2_best.pt and the prediction is made by using the CPU.

First, I tested an existing dataset image. The model predicted the Fully Ripe correctly with a softmax confidence score of 52.39%.

Then, I tested a new external image name as newoilpalm.jpg. The model predicted the images as Partially Ripe with 57.65% confidence.

The new image did not have and independently confirmed correct label. Thus, this demonstration only proves that the testing container can load the model and produce a prediction.

The testing container worked without the laptop's Conda environment.

```
(oilpalm) PS C:\degree\Sem_2\NEURAL NETWORK BASICS\Offline_Assignment> docker run --rm oilpalm-test:1.0
Testing environment ready
PyTorch: 2.14.0+cu126
scikit-learn: 1.9.1
```

```
(oilpalm) PS C:\degree\Sem_2\NEURAL NETWORK BASICS\Offline_Assignment> docker run --rm `
>>   --mount "type=bind,source=$($PWD.Path)\models,target=/app/models,readonly" `
>>   --mount "type=bind,source=$($PWD.Path)\demo_images,target=/app/demo_images,readonly" `
>>   oilpalm-test:1.0 `
>>   python src/docker_predict.py `
>>   --model /app/models/exp2_best.pt `
>>   --image /app/demo_images/newoilpalm.jpg
==========================================
OIL PALM FRUIT RIPENESS PREDICTION
==========================================
Image: /app/demo_images/newoilpalm.jpg
Device: cpu
Model dropout: 0.5
Predicted class: Partially Ripe
Confidence: 57.65%
==========================================
```

Figure 9: Docker testing environment and new image prediction.

6.3. Cross-Machine Verification

In order to check whether the CNN could run on another computer, I copied the testing files, trained model and sample image from my laptop to a desktop computer.

I built the Docker testing image on the desktop and ran the prediction using the CPU. The model predicted Partially Ripe with 57.65% confidence which was the same result as on the laptop.

This test showed that the saved CNN model could run on a different computer using Docker without installing PyTorch or conda directly on that computer.

```
PS C:\Users\User\Documents\cross_machine_test> docker run --rm `
>>   --mount "type=bind,source=$($PWD.Path)\models,target=/app/models,readonly" `
>>   --mount "type=bind,source=$($PWD.Path)\demo_images,target=/app/demo_images,readonly" `
>>   oilpalm-test:1.0 `
>>   python src/docker_predict.py `
>>   --model /app/models/exp2_best.pt `
>>   --image /app/demo_images/newoilpalm.jpg
==========================================
OIL PALM FRUIT RIPENESS PREDICTION
==========================================
Image: /app/demo_images/newoilpalm.jpg
Device: cpu
Model dropout: 0.5
Predicted class: Partially Ripe
Confidence: 57.65%
==========================================
PS C:\Users\User\Documents\cross_machine_test>
```

Figure 10: Cross-Machine Docker testing result on the desktop computer

7. Reflection

During this project, I faced several problems when setting up the Docker.

The first problem was that Docker Desktop kept showing “Starting the Docker Engine”. So, I checked the WSL status and I found that WSL was not installed yet. After installing the WSL 2 and checking that Ubuntu could run, I successfully started the Docker.

The second problem happened when I built the training Docker image. I received an externally-managed-environment error when installing the Python packages. Then, I solved this issues by adding --break-system-packages to the pip installation command in the Dockerfile. After that, I received another error which is the Dockerfile was missing the COPY requirements-train.txt . instruction. This issue is solved after I added the missing line and successfully built the training image.

After fixing the Docker problem, I successfully trained the CNN inside the training container. Moreover, I also used a separate testing container to load the saved model and predict a new image using the CPU.

Through these problems, I learned more about WSL 2, Dockerfiles, Python dependencies, bind mounts and Github permissions. Not for that, I also gained experience in checking error messages and fixing the problem on step at a time.

8. References

[1] I. Mustafa Kamal, “An Ordinal Dataset for Ripeness Level Classification in Oil Palm Fruit Quality Grading,” *Mendeley Data*, vol. 1, Nov. 2024, doi: 10.17632/424y96m6sw.1.

[2] “Bind Mounts,” *Docker Documentation*, 2024. https://docs.docker.com/engine/storage/bind-mounts/ (accessed Nov. 10, 2026).

Generative AI Use Declaration

ChatGPT was used in order to help to explain the programming concepts, troubleshoot errors, organize the report and improve English writing. The CNN training, experiments, evaluation and Docker testing was carried out using the project code. All the reported result were taken from the actual experiment record.